

問題2-4: ダウンサンプリング・QAM復調・デマッピングと BER

問題

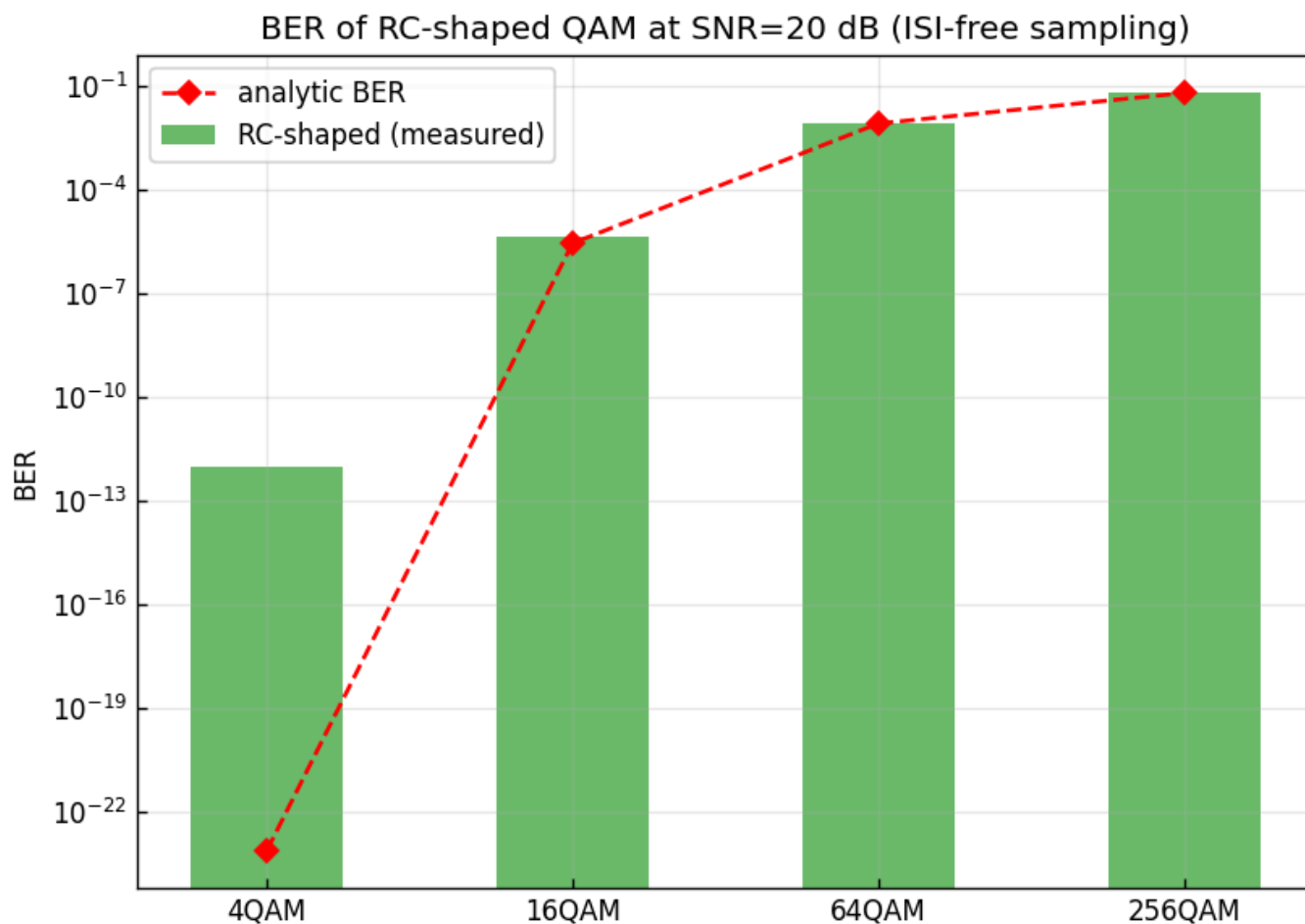
複素 AWGN が付加されたスペクトル整形 QAM サンプル列を、ISI が生じないタイミングでサンプル抽出(ダウンサンプリング)する。得られたサンプル列に QAM 復調(判定)+ QAMデマッピングを行って 復調ビット系列を求め、ビット誤り率(BER)を計算する。

入力にあたるサンプル列は問2-3 までで作ったもので、送信シンボルを 2 sample/symbol の raised cosine で 整形し、そこに複素 AWGN を載せたものになる。raised cosine は Nyquist パルスなので、判定点では整形なし (第1週)と同じ受信シンボルになり、BER も第1週・解析解と一致するはずである。この問題は、その一致を 実際の送受信チェーンで確かめるところにある。

実行結果(出力)

$\alpha=1.0$, 2 sps, シンボル数=500000, SNR=20.0 dB

方式	誤りビット数	BER(整形,実測)		BER(解析解)
4QAM	0	0.000e+00	7.620e-24	
16QAM	9	4.500e-06	2.904e-06	
64QAM	25025	8.342e-03	8.486e-03	
256QAM	262072	6.552e-02	6.517e-02	

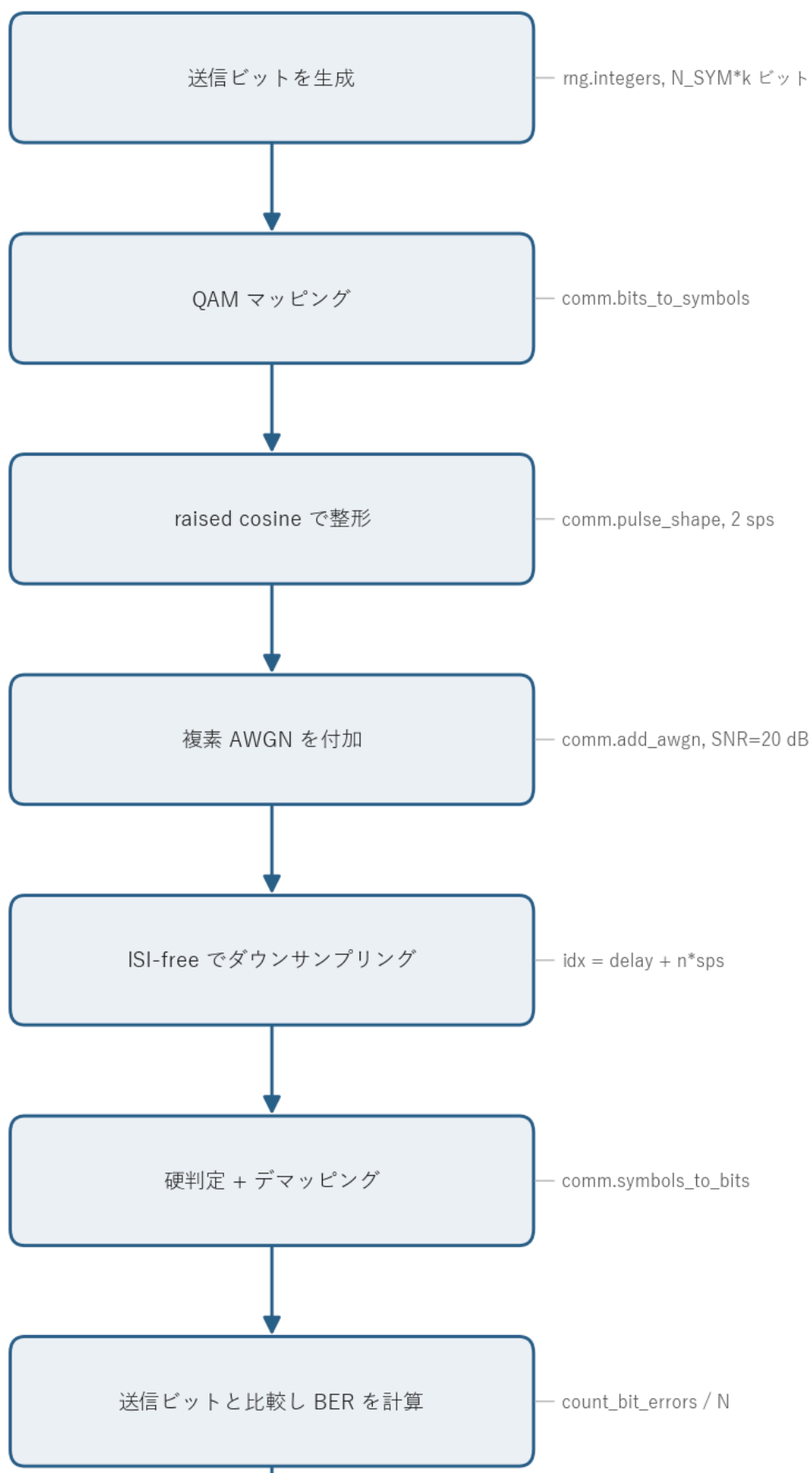


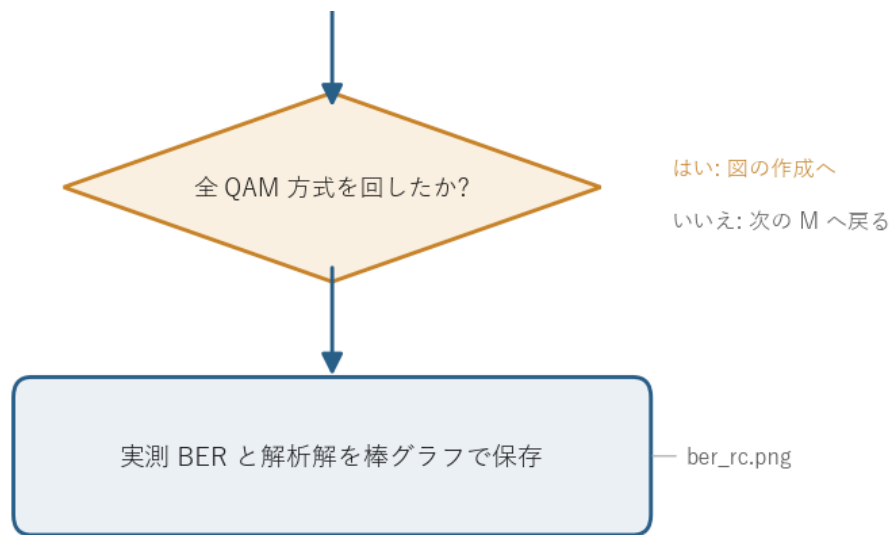
図の読み方 - 緑の棒が raised cosine 整形ありの実測 BER、赤◇が解析解。 - 縦軸は対数(BER は小さい桁まで動くため)。横軸は 4 つの QAM 方式。 - どの方式でも実測と解析解が重なる。整形しても判定点では理想点(1 sps)と同じ受信シンボルになるので、この結果になる。

0. 処理の流れ

`solution.py` を実行すると、4 つの QAM 方式(4/16/64/256)それぞれについて次の順で処理が進む。

整形QAMのダウンサンプリングとBERの処理フロー





前半(ビット生成からダウンサンプリングまで)が送受信チェーンの再現、後半(硬判定から BER)が復調と評価にあたる。送信ビットをそのまま手元に残しておき、復元したビットと突き合わせて誤り数を数える。分岐は「全 QAM 方式を回したか」の 1 箇所だけで、4 方式ぶんの BER がそろったら最後にまとめて棒グラフにする。

1. ダウンサンプリング(ISI-free 抽出)

「ダウンサンプリング」とは、2 sample/symbol の整形サンプル列から、シンボル中心(ISI が生じないタイミング) だけを取り出して 1 sample/symbol に戻すことである。

```
rx_sym = comm.downsample_isi_free(rx, delay, SPS, N_SYM) # idx = delay + n*sps
```

整形サンプル列は、各シンボルを `sps` 倍にアップサンプル(間に 0 を挿入)してから raised cosine と 畳み込んで作る。畳み込みでフィルタの群遅延 `delay = (len(h)-1)/2` ぶんだけ全体が後ろにずれるので、`n` 番目のシンボル中心は `delay + n*sps` の位置に来る。ここを 1 サンプルおき(`sps` 間隔)に拾えば、シンボルレート of 1 sample/symbol 列に戻る。

なぜシンボル中心では干渉が消えるのか

raised cosine は Nyquist の第一基準を満たすパルスで、インパルス応答 $h(t)$ が

$$h(0) = 1, \quad h(nT) = 0 \quad (n = \pm 1, \pm 2, \dots)$$

という形をしている(T はシンボル周期)。自分のシンボル中心では値が 1、ほかのシンボル中心では ちょうど 0 を通る。複数のシンボルを重ねて送っても、ある中心時刻で値を読むと、他シンボルのパルスはそこで 0 になっているので寄与しない。残るのは自分のシンボルだけになる。これが符号間干渉 (ISI,

inter-symbol interference)がゼロになる仕組みで、判定点での受信値は「元シンボル + 雑音」というきれいな形になる。

ロールオフ係数 α (コードの `BETA=1.0`)は、このゼロ点を保ったまま帯域を広げて時間応答の裾を速く減衰させるためのパラメータで、ゼロ点の位置そのものは α に依らず nT のままになる。だから α を変えても ISI-free のタイミングは変わらない。

2. 復調(QAM復調)+ デマッピング

ダウンサンプル後は第1週とまったく同じ処理になる。

- **QAM復調(硬判定):** 各受信シンボルを最近傍コンステレーション点へ丸める(問1-4)。
- **QAMデマッピング:** 判定したシンボルをグレイ符号の逆変換でビットに戻す(問1-5)。

共通ライブラリでは `comm.symbols_to_bits` がこの2つをまとめて行う。方形 QAM は I 軸(実部)と Q 軸(虚部)が独立な $\sqrt{M}$ 値 PAM なので、判定は各軸で「最も近い振幅レベルはどれか」を求めるだけになる。得られたビットを送信ビットと比べて BER を計算する。

3. なぜ整形しても BER が変わらないのか

これがこの問題の最重要点になる。結果を見ると、整形あり(本問)の BER は、整形なし(第1週・問1-6)や解析解とほぼ一致している。理由は2つの性質の組み合わせにある。

1. **Nyquist 性(ゼロ ISI):** シンボル中心では信号は元シンボルそのもの(電力 1)。整形による干渉が無い。
2. **白色雑音:** AWGN は各サンプル独立で、どの周波数にも一様にエネルギーを持つ。ダウンサンプリングで一部のサンプルを捨てても、残った各サンプルの雑音電力は変わらず N_0 (SNR=20 dB なら 0.01)のままになる。

したがって判定点での受信シンボルは「元シンボル + 電力 0.01 の複素ガウス雑音」で、1 sps のときと統計的に同一になる。判定の前段(整形と中心抽出)が信号も雑音もそのまま素通しさせるので、誤りの起きやすさは変わらず、BER も同じになる。

方式	BER(整形あり, 本問)	BER(整形なし, 問1-6)	解析解
16QAM	4.5×10^{-6}	2.8×10^{-6}	2.9×10^{-6}
64QAM	8.34×10^{-3}	8.43×10^{-3}	8.49×10^{-3}
256QAM	6.55×10^{-2}	6.55×10^{-2}	6.52×10^{-2}

16QAM の実測がばらつくのは、BER が小さく誤り数が一桁(9 個)なので統計的に揺らぐためである。誤り数が小さいほど、母数(送信ビット数)に対する比率の相対誤差は大きくなる。4QAM は SNR=20 dB では理論 BER が 10^{-24} 台で、50 万シンボルでは 1 個も誤りが出ない(実測 0)。

整形の意味は波形にある。整形は BER を改善するためではなく、実際に送れる帯域内に信号を収め、隣接チャネルへの漏れを抑え、サンプリング定理を満たす連続波形を作るためにある。理想条件下では BER は変わらないが、帯域制限のある実システムでは整形が必須になる。

4. 関数ごとの詳細

`solution.py` は `main()` 1 つで、送受信の各段を `comm` の関数呼び出しでつないでいる。チェーンの順に、使っている共通関数を引数・戻り値・内部処理で見えていく。

`comm.bits_to_symbols(bits, M)`

ビット列を規格化複素 QAM シンボル列に変換する(QAMマッピング)。

項目	内容
引数	<code>bits</code> : 0/1 のビット列。長さは $\log_2 M$ の倍数。 <code>M</code> : QAM 多値数(4/16/64/256)。
戻り値	長さ <code>len(bits)/k</code> の規格化複素シンボル列(平均電力 1)。

内部の流れ。

- `qam_params(M)` で 1 シンボルのビット数 $k = \log_2 M$ 、軸あたりレベル数 $L = \sqrt{M}$ 、軸あたりビット数 $kk = k/2$ を求める。
- ビット列を `(-1, k)` 行列に並べ替え、前半 kk ビットを I 軸、後半 kk ビットを Q 軸のグレイ符号として読む。
- グレイ符号を `_gray_decode` で振幅順のレベル番号 `0..L-1` に戻す。
- レベル番号を振幅 $2 \cdot \text{idx} - (L - 1)$ ($\{-(L - 1), \dots, L - 1\}$ の奇数列)に変換し、`qam_norm(M)` で割って平均電力を 1 に規格化する。

`qam_norm(M)` が $\sqrt{2(M-1)/3}$ なのは、レベル $\{\pm 1, \pm 3, \dots\}$ の方形 QAM の平均電力が $2(M-1)/3$ だからで、この値で割ると平均 $|s|^2 = 1$ になる。

`comm.pulse_shape(sym, sps, beta, span)`

シンボル列を raised cosine で整形する。

項目	内容
引数	<code>sym</code> : シンボル列。 <code>sps</code> : 1 シンボルあたりサンプル数(2)。 <code>beta</code> : ロールオフ α (1.0)。 <code>span</code> : フィルタ長(シンボル数, 12)。
戻り値	(<code>shaped, delay</code>)。 <code>shaped</code> は整形サンプル列、 <code>delay</code> は群遅延 [サンプル]。

内部の流れ。

1. `rc_filter(beta, sps, span)` で raised cosine の FIR 係数 $\mathbf{h}$ を作る。全長は `span*sps+1` 点。
2. `up = np.zeros(len(sym)*sps)` を用意し、 `up[::sps] = sym` で `sps` サンプルおきにシンボルを置く(間は 0)。これがアップサンプリング。
3. `shaped = np.convolve(up, h)` で畳み込み、各シンボルにパルス波形を載せる。
4. `delay = (len(h)-1)//2` を群遅延として返す。 `shaped[delay + n*sps]` が n 番目のシンボル中心になる。

`up[::sps] = sym` の 1 行が「シンボルレートからサンプルレートへの引き伸ばし」を担う。間に 0 を挟むことで、畳み込み後に各シンボルがちょうど `sps` サンプル間隔で並ぶ。

```
comm.add_awgn(sym, snr_db, signal_power=1.0, rng=rng)
```

複素 AWGN(白色ガウス雑音)を SNR [dB] 指定で付加する。

項目	内容
引数	<code>sym</code> : 複素信号。 <code>snr_db</code> : 信号電力対雑音電力比 [dB]。 <code>signal_power</code> : 信号電力(本間は 1.0 を明示)。 <code>rng</code> : 乱数生成器。
戻り値	雑音付加後の複素信号(同じ長さ)。

内部の流れ。

1. `snr_lin = 10**(snr_db/10)` で dB を真数に直す。
2. `noise_power = signal_power / snr_lin` が複素雑音の総電力 N_0 (SNR=20 dB なら 0.01)。
3. `sigma = sqrt(noise_power/2)` を 1 軸あたりの標準偏差にし、実部・虚部に独立な正規乱数を生成して加える。

総電力を実部・虚部に半分ずつ分けるのは、複素雑音 $\mathbf{n} = \mathbf{n}_I + j\mathbf{n}_Q$ の電力が $E[|\mathbf{n}|^2] = \sigma_I^2 + \sigma_Q^2$ で、これを N_0 にそろえるためである。 `signal_power=1.0` を渡しているのは、整形後のサンプル列から実測するとアップサンプルの 0 が混じって平均電力が下がるので、規格化済みのシンボル電力 1 を基準に SNR を定義するためになる。

`comm.downsample_isi_free(shaped, delay, sps, n_sym)`

整形サンプル列から ISI の生じないシンボル中心だけを抜き出す。

項目	内容
引数	<code>shaped</code> : 整形サンプル列。 <code>delay</code> : 群遅延。 <code>sps</code> : サンプル/シンボル。 <code>n_sym</code> : 取り出すシンボル数。
戻り値	長さ最大 <code>n_sym</code> の 1 sample/symbol シンボル列。

処理は 2 行。 `idx = delay + np.arange(n_sym)*sps` でシンボル中心のインデックスを作り、配列範囲を超えるものを `idx[idx < len(shaped)]` で落としてから `shaped[idx]` を返す。 `delay` の足し込みが 畳み込みでずれたぶんの補正で、これを忘れると中心からずれた点を拾って ISI が混入する。

`comm.symbols_to_bits(sym, M)`

受信シンボル列を硬判定してビット列に戻す(復調 + デマッピング)。

項目	内容
引数	<code>sym</code> : 受信シンボル列(雑音を含んでよい)。 <code>M</code> : QAM 多値数。
戻り値	0/1 のビット列(長さ <code>len(sym)*k</code>)。

内部の流れ。

1. `_slice_indices` でシンボルを `qam_norm(M)` 倍して整数振幅スケールに戻し、各軸で最近傍レベル番号 $0..L-1$ に丸める (`np rint + np.clip` で端をはみ出さないよう抑える)。これが硬判定。
2. レベル番号を `_gray_encode` でグレイ符号に直し、`_int_to_bits` で MSB first のビットに展開する。
3. I 軸ビットと Q 軸ビットを連結して 1 シンボルぶんのビット列にし、全体を 1 次元に並べて返す。

判定が「最近傍レベルへの丸め」で済むのは、方形 QAM の I/Q が独立な等間隔 PAM だからで、2 次元の最近傍探索を 2 つの 1 次元の丸めに分解できる。

`comm.count_bit_errors(tx_bits, rx_bits)` と `comm.ber_theory_qam(M, snr_db)`

`count_bit_errors` は送信・受信ビットの不一致数を `np.count_nonzero(tx != rx)` で数える。BER はこれを比較したビット数で割った値になる。

`ber_theory_qam` はグレイ符号化方形 M-QAM の BER 解析解(近似式)

$$\text{BER} \approx \frac{4}{k} \left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3}{M-1}} \gamma_s\right)$$

を返す。 γ_s は 1 シンボルあたりの SNR(E_s/N_0)、 $Q(x) = \frac{1}{2} \text{erfc}(x/\sqrt{2})$ 。実測と この式を並べて、整形ありでも解析解どおりになることを確認している。

`main()`

全体をつなぐ関数。

1. 乱数シードを固定し、ヘッダを表示する。
2. 4 つの QAM 方式について、ビット生成 → マッピング → 整形 → AWGN → ダウンサンプリング → 判定 + デマッピング → BER の順に処理し、実測 BER と解析解を `results` に貯める。
3. 全方式の結果を対数軸の棒グラフ(実測)+ 折れ線(解析解)にし、`ber_rc.png` に保存する。図のラベルは日本語フォントに依存しないよう英語にしている。

5. コードのポイント

- 送受信チェーン: `bits → bits_to_symbols → pulse_shape → add_awgn → downsample_isi_free → symbols_to_bits`。
- 送信ビットを基準に BER を計算(`comm.count_bit_errors`)。長さがずれても短い方に合わせる。
- BER は対数軸の棒グラフで解析解と比較。棒の最小値を `1e-12` で下限クリップしているのは、実測 0(4QAM)でも対数軸に描けるようにするため。

実行

`python solution.py`

標準出力は冒頭の「実行結果(出力)」、生成図は `ber_rc.png` を参照。処理フロー図 `flow.png` は `python make_flow.py` で作り直せる。

6. この問題のつながり

次の問2-5 では、この整形ありのチェーンで BER の SNR 依存性(問2-1 と同じ曲線)を描き、整形しても解析解どおりの曲線になることを確認する。本問が「ある 1 点の SNR での一致」を示したのに対し、問2-5 は それを SNR を振った曲線全体に広げたものにあたる。